

closest pair-1

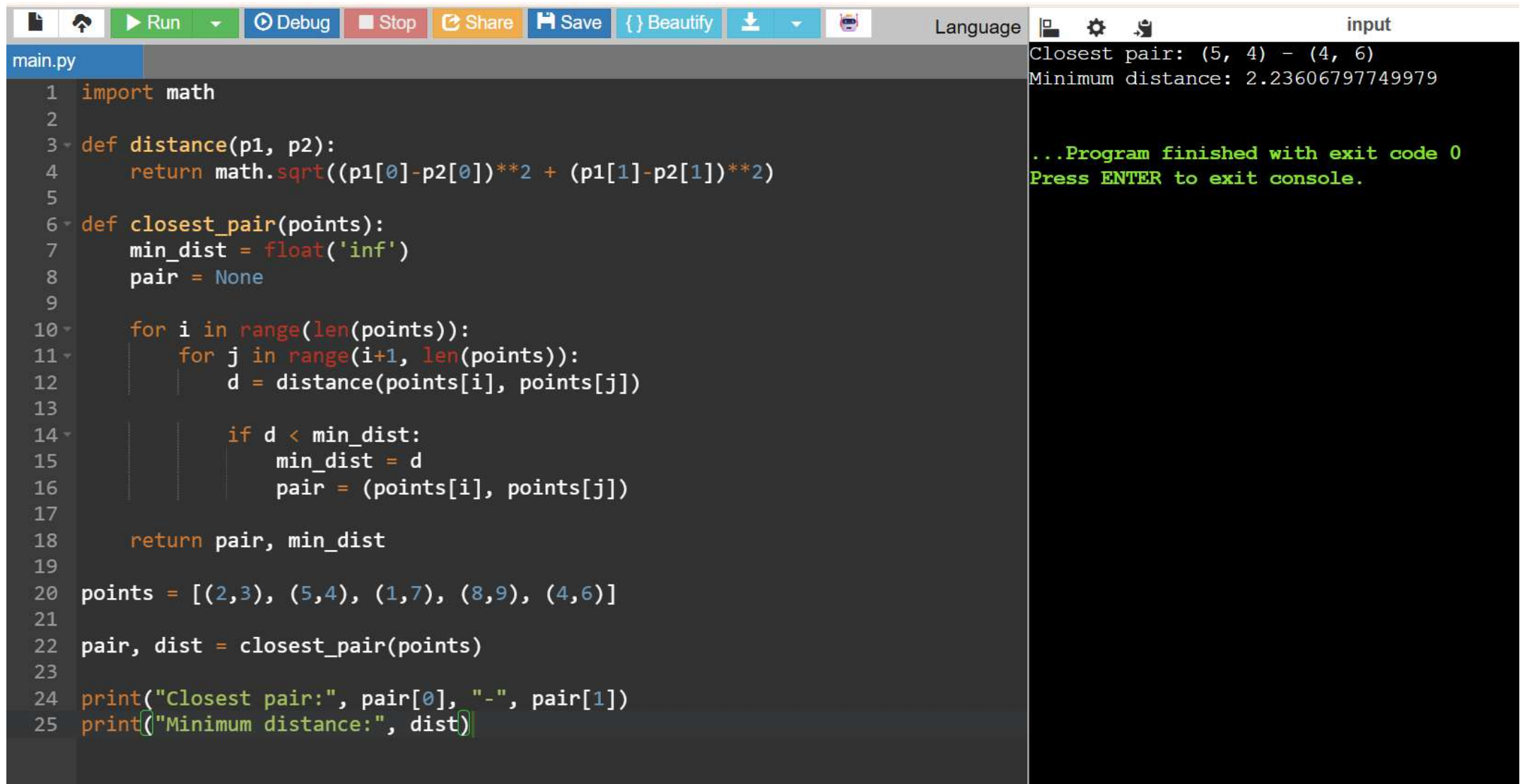
main.py

```
1 import math
2
3 def distance(p1, p2):
4     return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
5
6 def closest_pair(points):
7     min_dist = float('inf')
8     pair = None
9
10    for i in range(len(points)):
11        for j in range(i+1, len(points)):
12            d = distance(points[i], points[j])
13            if d < min_dist:
14                min_dist = d
15                pair = (points[i], points[j])
16
17    return pair, min_dist
18
19 points = [(1,2), (4,5), (7,8), (3,1)]
20
21 pair, dist = closest_pair(points)
22
23 print("Closest pair:", pair[0], "-", pair[1])
24 print("Minimum distance:", dist)
```

Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.23606797749979

...Program finished with exit code 0
Press ENTER to exit console.

closest pair-2



```
1 import math
2
3 def distance(p1, p2):
4     return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
5
6 def closest_pair(points):
7     min_dist = float('inf')
8     pair = None
9
10    for i in range(len(points)):
11        for j in range(i+1, len(points)):
12            d = distance(points[i], points[j])
13
14            if d < min_dist:
15                min_dist = d
16                pair = (points[i], points[j])
17
18    return pair, min_dist
19
20 points = [(2,3), (5,4), (1,7), (8,9), (4,6)]
21
22 pair, dist = closest_pair(points)
23
24 print("Closest pair:", pair[0], "-", pair[1])
25 print("Minimum distance:", dist)
```

Closest pair: (5, 4) - (4, 6)
Minimum distance: 2.23606797749979

...Program finished with exit code 0
Press ENTER to exit console.

closest pair-3

```
File Edit View Shell Run Debug Stop Share Save Beautify Download Language input
1 import math
2
3 def distance(p1, p2):
4     return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
5
6 def closest_pair(points):
7     min_dist = float('inf')
8     pair = None
9
10    for i in range(len(points)):
11        for j in range(i+1, len(points)):
12            d = distance(points[i], points[j])
13
14            if d < min_dist:
15                min_dist = d
16                pair = (points[i], points[j])
17
18    return pair, min_dist
19
20 points = [(0,0), (2,2), (3,1), (6,5)]
21
22 pair, dist = closest_pair(points)
23
24 print("Closest pair:", pair[0], "-", pair[1])
25 print("Minimum distance:", dist)
```

Closest pair: (2, 2) - (3, 1)
Minimum distance: 1.4142135623730951

...Program finished with exit code 0
Press ENTER to exit console.

closest pair-4

Click to go forward, hold to see history

StopShareSaveBeautify

Language

main.py

```
1 import math
2
3 def distance(p1, p2):
4     return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
5
6 def closest_pair(points):
7     min_dist = float('inf')
8     pair = None
9
10    for i in range(len(points)):
11        for j in range(i+1, len(points)):
12            d = distance(points[i], points[j])
13
14            if d < min_dist:
15                min_dist = d
16                pair = (points[i], points[j])
17
18    return pair, min_dist
19
20 points = [(10,20), (15,25), (30,40), (12,22)]
21
22 pair, dist = closest_pair(points)
23
24 print("Closest pair:", pair[0], "-", pair[1])
25 print("Minimum distance:", dist)
```

input

```
Closest pair: (10, 20) - (12, 22)
Minimum distance: 2.8284271247461903

...Program finished with exit code 0
Press ENTER to exit console.
```

closest pair-5

```
main.py
1 import math
2
3 def distance(p1, p2):
4     return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
5
6 def closest_pair(points):
7     min_dist = float('inf')
8     pair = None
9
10    for i in range(len(points)):
11        for j in range(i+1, len(points)):
12            d = distance(points[i], points[j])
13
14            if d < min_dist:
15                min_dist = d
16                pair = (points[i], points[j])
17
18    return pair, min_dist
19
20 points = [(10,0),(11,5),(5,3),(9,3.5),(15,3),(12.5,7),(6,6.5),(7.5,4.5)]
21
22 pair, dist = closest_pair(points)
23
24 print("Closest Pair:", pair[0], "-", pair[1])
25 print("Minimum Distance:", dist)
26
27 print("\nTime Complexity: O(n^2)")
28 print("Space Complexity: O(1)")
```

Closest Pair: (9, 3.5) - (7.5, 4.5)
Minimum Distance: 1.8027756377319946

Time Complexity: O(n^2)
Space Complexity: O(1)

...Program finished with exit code 0
Press ENTER to exit console.

convex hull -1

Click to go forward, hold to see history

Stop Share Save {} Beautify

Language

input

main.py

```
6     return points
7
8     hull = []
9
10    left = min(range(len(points)), key=lambda i: points[i][0])
11
12    p = left
13    while True:
14        hull.append(points[p])
15
16        q = (p + 1) % len(points)
17
18        for i in range(len(points)):
19            if orientation(points[p], points[i], points[q]) < 0:
20                q = i
21
22        p = q
23
24        if p == left:
25            break
26
27    return hull
28
29    points = [(0,0), (1,1), (2,2), (0,2), (2,0)]
30
31    print("Convex Hull Points:")
32    for p in convex_hull(points):
33        print(p)
```

Convex Hull Points:
(0, 0)
(2, 0)
(2, 2)
(0, 2)

...Program finished with exit code 0
Press ENTER to exit console.

convex hull-2

```
main.py
6     return points
7
8     hull = []
9     left = min(range(len(points)), key=lambda i: points[i][0])
10
11     p = left
12
13     while True:
14         hull.append(points[p])
15
16         q = (p + 1) % len(points)
17
18         for i in range(len(points)):
19             if orientation(points[p], points[i], points[q]) < 0:
20                 q = i
21
22         p = q
23
24         if p == left:
25             break
26
27     return hull
28
29 points = [(1,2), (3,1), (5,3), (4,6), (2,5)]
30
31 print("Convex Hull Points:")
32 for p in convex_hull(points):
33     print(p)
```

input

Convex Hull Points:
(1, 2)
(3, 1)
(5, 3)
(4, 6)
(2, 5)

...Program finished with exit code 0
Press ENTER to exit console.

convex hull -3

```
main.py
9 hull = []
10
11 left = min(range(len(points)), key=lambda i: points[i][0])
12
13 p = left
14
15 while True:
16     hull.append(points[p])
17
18     q = (p + 1) % len(points)
19
20     for i in range(len(points)):
21         if orientation(points[p], points[i], points[q]) < 0:
22             q = i
23
24     p = q
25
26     if p == left:
27         break
28
29 return hull
30
31
32 points = [(0,3), (1,1), (2,2), (4,4), (3,0)]
33
34 print("Convex Hull Points:")
35 for p in convex_hull(points):
36     print(p)
```

input

Convex Hull Points:
(0, 3)
(1, 1)
(3, 0)
(4, 4)

...Program finished with exit code 0
Press ENTER to exit console.

convex hull-4

```
main.py
9 hull = []
10
11 left = min(range(len(points)), key=lambda i: points[i][0])
12
13 p = left
14
15 while True:
16     hull.append(points[p])
17
18     q = (p + 1) % len(points)
19
20     for i in range(len(points)):
21         if orientation(points[p], points[i], points[q]) < 0:
22             q = i
23
24     p = q
25
26     if p == left:
27         break
28
29 return hull
30
31 points = [(2,2), (4,1), (6,3), (5,5), (3,6), (1,4)]
32
33 print("Convex Hull Points:")
34 for p in convex_hull(points):
35     print(p)
```

input

Convex Hull Points:
(1, 4)
(2, 2)
(4, 1)
(6, 3)
(5, 5)
(3, 6)

...Program finished with exit code 0
Press ENTER to exit console.

convex hull-5

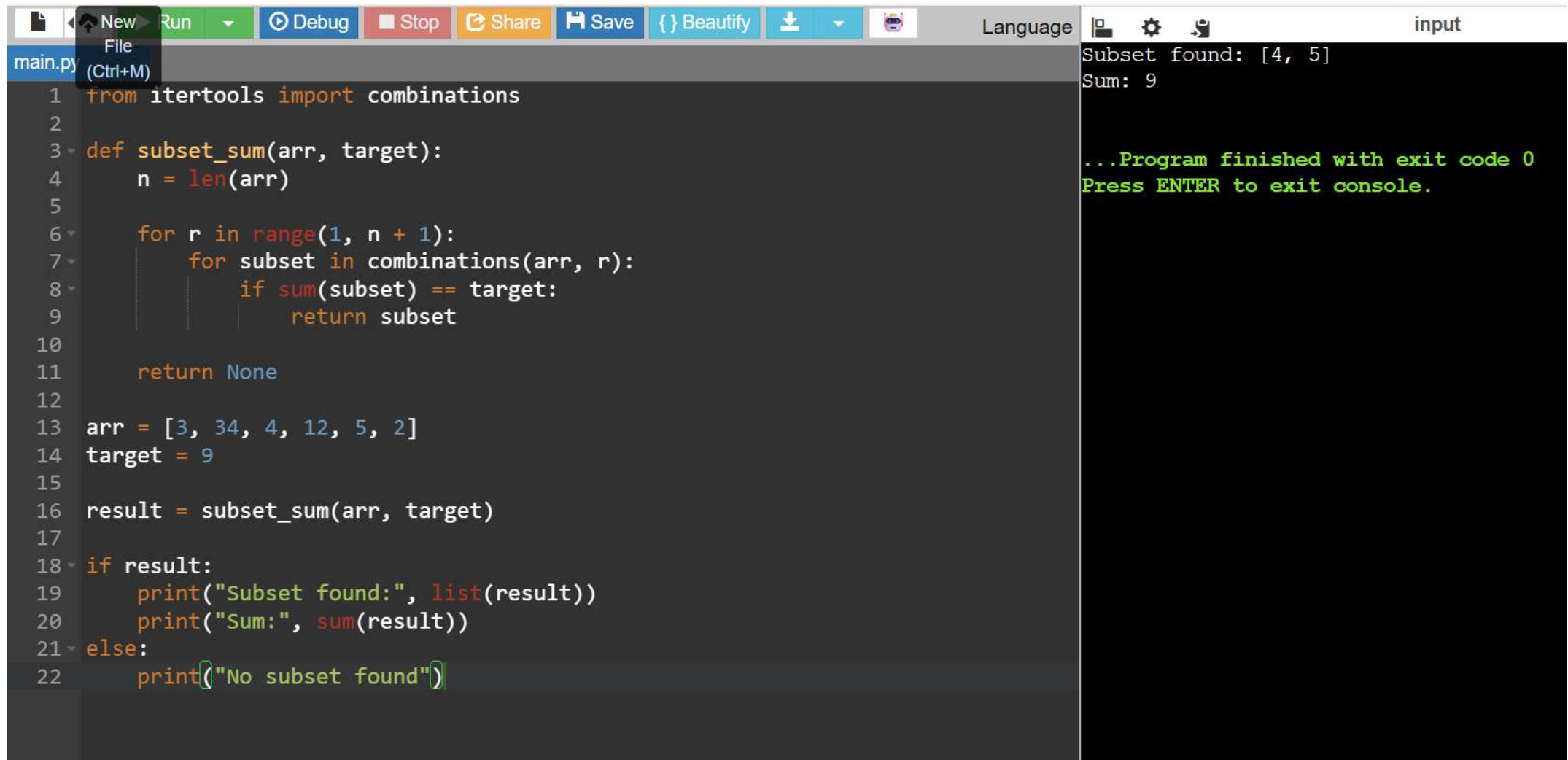
```
main.py
9 hull = []
10
11 left = min(range(len(points)), key=lambda i: points[i][0])
12
13 p = left
14
15 while True:
16     hull.append(points[p])
17
18     q = (p + 1) % len(points)
19
20     for i in range(len(points)):
21         if orientation(points[p], points[i], points[q]) < 0:
22             q = i
23
24     p = q
25
26     if p == left:
27         break
28
29 return hull
30
31
32 points = [(1,1), (2,4), (5,5), (6,2), (3,0)]
33
34 print("Convex Hull Points:")
35 for p in convex_hull(points):
36     print(p)
```

input

Convex Hull Points:
(1, 1)
(3, 0)
(6, 2)
(5, 5)
(2, 4)

...Program finished with exit code 0
Press ENTER to exit console.

search subset-1



The image shows a code editor window with a Python script and its console output. The code defines a function `subset_sum` that finds a subset of an array with a target sum. The array is `[3, 34, 4, 12, 5, 2]` and the target is `9`. The output shows the subset `[4, 5]` and the sum `9`.

```
main.py
1 from itertools import combinations
2
3 def subset_sum(arr, target):
4     n = len(arr)
5
6     for r in range(1, n + 1):
7         for subset in combinations(arr, r):
8             if sum(subset) == target:
9                 return subset
10
11     return None
12
13 arr = [3, 34, 4, 12, 5, 2]
14 target = 9
15
16 result = subset_sum(arr, target)
17
18 if result:
19     print("Subset found:", list(result))
20     print("Sum:", sum(result))
21 else:
22     print("No subset found")
```

input

Subset found: [4, 5]
Sum: 9

...Program finished with exit code 0
Press ENTER to exit console.

search subset-2

```
main.py
9 min_distance = float('inf')
10 best_path = None
11
12 for perm in permutations(range(1, n)):
13     path = (0,) + perm
14
15     total = 0
16     for i in range(n-1):
17         total += distance(cities[path[i]], cities[path[i+1]])
18
19     total += distance(cities[path[-1]], cities[path[0]])
20
21     if total < min_distance:
22         min_distance = total
23         best_path = path
24
25 final_path = [cities[i] for i in best_path]
26 final_path.append(cities[0])
27
28 return min_distance, final_path
29
30 # Test Case 1
31 cities = [(1,2), (4,5), (7,1), (3,6)]
32
33 dist, path = tsp(cities)
34
35 print("Shortest Distance:", dist)
36 print("Shortest Path:", path)
```

Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6)]

...Program finished with exit code 0
Press ENTER to exit console.

Airplane mode on

search subset-3

```
main.py
10 n = len(cost_matrix)
11
12 min_cost = float('inf')
13 best_assignment = None
14
15 for assignment in permutations(range(n)):
16     cost = total_cost(assignment, cost_matrix)
17
18     if cost < min_cost:
19         min_cost = cost
20         best_assignment = assignment
21
22     return best_assignment, min_cost
23
24 cost_matrix = [
25     [3,10,7],
26     [8,5,12],
27     [4,6,9]
28 ]
29
30 assignment, cost = assignment_problem(cost_matrix)
31
32 print("Optimal Assignment")
33
34 for worker, task in enumerate(assignment):
35     print("Worker", worker+1, "-> Task", task+1)
36
37 print("Total Cost:", cost)
```

Optimal Assignment
Worker 1 -> Task 3
Worker 2 -> Task 2
Worker 3 -> Task 1
Total Cost: 16

...Program finished with exit code 0
Press ENTER to exit console.

search subset-4

Run

Debug

Stop

Share

Save

Beautify

Language

input

Upload

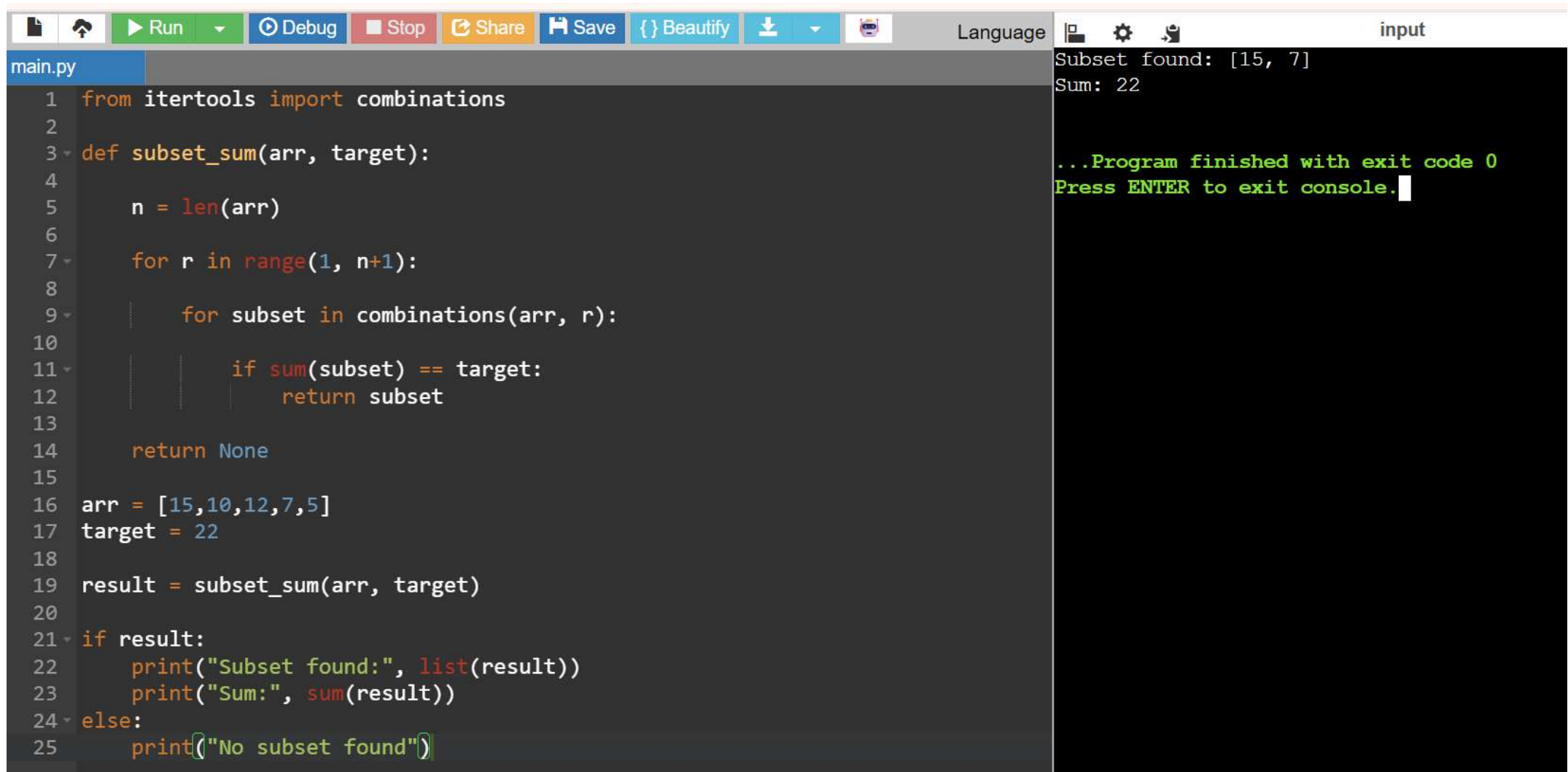
File

```
9 def knapsack(weights, values, capacity):
10     n = len(weights)
11
12     best = []
13     max_value = 0
14
15     for r in range(n+1):
16         for subset in combinations(range(n), r):
17
18             if is_feasible(subset, weights, capacity):
19
20                 value = total_value(subset, values)
21
22                 if value > max_value:
23                     max_value = value
24                     best = subset
25
26     return best, max_value
27
28 weights = [2,3,1]
29 values = [4,5,3]
30 capacity = 4
31
32 items, value = knapsack(weights, values, capacity)
33
34 print("Optimal Selection:", list(items))
35 print("Total Value:", value)
```

Optimal Selection: [1, 2]
Total Value: 8

...Program finished with exit code 0
Press ENTER to exit console.

search subset-5



The image shows a code editor window with a dark theme. The top toolbar includes icons for file operations, a 'Run' button, a 'Debug' button, a 'Stop' button, a 'Share' button, a 'Save' button, a 'Beautify' button, and a dropdown menu. The file name 'main.py' is visible in the top left. The code is a Python script that uses the 'itertools' module to find a subset of an array that sums to a target value. The array is [15, 10, 12, 7, 5] and the target is 22. The script finds the subset [15, 7] and prints its sum as 22. The output console on the right shows the same results and a message indicating the program finished with exit code 0.

```
1 from itertools import combinations
2
3 def subset_sum(arr, target):
4     n = len(arr)
5     for r in range(1, n+1):
6         for subset in combinations(arr, r):
7             if sum(subset) == target:
8                 return subset
9
10    return None
11
12 arr = [15,10,12,7,5]
13 target = 22
14
15 result = subset_sum(arr, target)
16
17 if result:
18     print("Subset found:", list(result))
19     print("Sum:", sum(result))
20 else:
21     print("No subset found")
```

Subset found: [15, 7]
Sum: 22

...Program finished with exit code 0
Press ENTER to exit console.